Linear Regression Using Gradient Descent & Stochastic Gradient Descent

A First-Principles Comparative Study on Optimization Dynamics & Generalization Parity

PROJECT INVESTIGATORS

Sakhunala Sahaja	25WU0102237
Sufiyan Yusuf Jahan	25WU0102276
Sai Sri Charan Chuduri	25WU0102235

ACADEMIC DETAILS

COURSE

Foundations of Artificial Intelligence

FACULTY SUPERVISOR

Dr. YVR Naga Pavan

INSTITUTION

School of Technology, Woxsen University

Submission Date: October 11, 2026 | Academic Term: 2026

GitHub Repository: https://github.com/sahajasakhunala/linear-regression-gd-sgd

1. Abstract

Optimization algorithms constitute the mathematical engine of modern artificial intelligence and statistical learning [1], [2]. In this project, an end-to-end empirical and theoretical comparative study between Batch Gradient Descent (GD) and Stochastic Gradient Descent (SGD) is conducted for univariate Linear Regression. Built entirely from first mathematical principles using Python and NumPy [5], the system eliminates reliance on high-level library estimators (such as scikit-learn's `LinearRegression` or `SGDRegressor`) in order to make every intermediate computational step transparent: forward hypothesis generation, Mean Squared Error (MSE) loss computation, closed-form analytical gradient calculation, and iterative parameter updates.

The experiment is conducted on a canonical 15-observation academic dataset tracking the relationship between study hours and exam scores. Methodological rigor is maintained via an isolated 70/30 train/test partition (10 training observations, 5 held-out test observations) under seed 42 with zero data leakage. Both iterative optimizers are evaluated against the closed-form Ordinary Least Squares (OLS) analytical reference [3]. The experimental results demonstrate that both algorithms successfully recover the underlying data-generating relationship: Batch GD converges to $\hat{y} = 7.7822x + 28.2121$, while SGD converges to $\hat{y} = 7.7156x + 28.5394$. On unseen test data, both models achieve identical generalization fidelity (test $R^2 = 0.9973$, test RMSE ≈ 0.93 marks). Crucially, SGD achieves equivalent predictive accuracy while requiring an order of magnitude fewer individual sample evaluations (1,000 versus 10,000 for Batch GD), illustrating fundamental trade-offs between deterministic and stochastic optimization [2], [4].

2. Problem Statement

Contemporary machine learning practice frequently treats model fitting as an opaque API invocation. While abstractions accelerate rapid prototyping, they obscure the underlying mechanics of optimization: loss landscape topology, learning rate sensitivity, gradient variance, and parameter trajectory stability [1], [4]. When optimization is treated as a black-box, practitioners miss foundational engineering insights into how individual data points guide parameter updates, why stochastic updates introduce variance, and how data leakage distorts empirical validation [2].

To resolve this pedagogical challenge, this project develops a transparent, from-scratch implementation of Linear Regression. By deriving gradients analytically, implementing update rules manually, maintaining strict test-set isolation, and comparing Batch GD against SGD, this study provides complete clarity into how Linear Regression learns from data.

3. Introduction

Supervised regression identifies continuous functional mappings between input features and target outcomes. In its univariate formulation, Linear Regression represents the bedrock foundation of statistical learning and predictive modeling [1], [3]:

The Univariate Linear Hypothesis `Equation 1`

$$\hat{y}_i = w \cdot x_i + b$$

Where $\mathbf{x_i}$ denotes study hours for student i, $\hat{\mathbf{y}}_\mathbf{i}$ denotes the predicted exam score, **w** denotes the slope/weight (rate of score gain per hour), and **b** denotes the baseline intercept score at zero study hours.

While small linear systems can be solved directly via the Normal Equation / Ordinary Least Squares closed-form solution [3], iterative first-order optimization via Gradient Descent forms the universal backbone for training deep neural networks, transformers, and large-scale artificial intelligence architectures where analytical matrix inversion is computationally intractable [2].

4. Project Objectives

- **Mathematical Formulation:** Formulate the linear regression hypothesis and derive analytical partial derivatives of Mean Squared Error (MSE) with respect to weight (w) and bias (b) [1].
- **First-Principles Implementation:** Develop Batch GD and SGD completely from scratch in Python [8] using vectorized NumPy routines [5], avoiding black-box library fits.
- **Zero Data Leakage Safeguards:** Partition data into 70% training (n=10) and 30% testing (n=5) subsets with strict isolation during fitting [3].
- **Comparative Trajectory Tracking:** Record iteration-by-iteration and epoch-by-epoch loss trajectories to contrast deterministic versus stochastic convergence [4].
- **Comprehensive Generalization Evaluation:** Evaluate models on unseen test observations using MSE, RMSE, and R^2 alongside closed-form OLS solutions.
- **Publication-Quality Visualization:** Generate six high-resolution visual figures illustrating dataset partition, fitted lines, loss curves, and actual versus predicted test scores using Matplotlib [7].

5. Theoretical Background & Mathematical Derivations

To make the mathematics intuitive and structured, all equations, variables, and derivations are organized systematically below following standard statistical learning principles [1], [3].

5.1 Master Variable Definition Reference

Symbol	Mathematical Meaning	Project Physical Interpretation	Experimental Value / Domain
x_i	Independent Feature	Hours studied by student i	$1.0 \le x \le 8.0$ hours
y_i	Ground Truth Target	Actual exam percentage score	$35.0 \le y \le 89.0$ marks
$\hat{y}_i$	Hypothesis Output	Model predicted exam score: $\hat{y}_i = wx_i + b$	Continuous scalar (marks)
w	Weight Parameter (Slope)	Score gained per study hour logged	Initialized at 0.0; Converges ~7.78
b	Bias Parameter (Intercept)	Baseline exam score with zero study hours	Initialized at 0.0; Converges ~28.21
n	Sample Size	Number of observations in training batch	n = 10 training observations
α (alpha)	Learning Rate	Optimization step size multiplier	Fixed hyperparameter $\alpha = 0.01$
J(w, b)	Objective Loss Function	Mean Squared Error (MSE) across dataset	Convex quadratic surface

5.2 Mean Squared Error (MSE) Objective Function

Mean Squared Error (MSE) Formula Equation 2

$$J(w, b) = 1/n \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = 1/n \sum_{i=1}^{n} (y_i - (w \cdot x_i + b))^2$$

Why MSE is selected [1], [3]: (1) *Strict Convexity* guarantees a unique global minimum with zero local minima traps; (2) *Continuous Differentiability* produces smooth analytical gradients everywhere; (3) *Quadratic Penalty* strongly discourages large residual errors; (4) Minimizing MSE is mathematically equivalent to Maximum Likelihood Estimation (MLE) under Gaussian error assumptions.

5.3 Step-by-Step Analytical Gradient Derivations

The gradient vector $\nabla J(w, b)$ is computed by taking partial derivatives of $J(w, b)$ using the calculus chain rule [2]:

Step 1: Weight Gradient Derivation ($\partial J / \partial w$) Equation 3a

1. Apply chain rule: $\partial J/\partial w = 1/n \sum_{i=1}^{n} 2(y_i - \hat{y}_i) \cdot \partial/\partial w (y_i - w \cdot x_i - b)$
2. Substitute inner derivative: $\partial/\partial w (y_i - w \cdot x_i - b) = -x_i$
3. Factor constants: $\partial J/\partial w = -2/n \sum_{i=1}^{n} x_i \cdot (y_i - \hat{y}_i)$

Step 2: Bias Gradient Derivation ($\partial J / \partial b$) Equation 3b

1. Apply chain rule: $\partial J/\partial b = 1/n \sum_{i=1}^{n} 2(y_i - \hat{y}_i) \cdot \partial/\partial b (y_i - w \cdot x_i - b)$
2. Substitute inner derivative: $\partial/\partial b (y_i - w \cdot x_i - b) = -1$
3. Factor constants: $\partial J/\partial b = -2/n \sum_{i=1}^{n} (y_i - \hat{y}_i)$

5.4 Master Comparison: Batch GD vs. Stochastic GD Equations

The mathematical formulation of parameter updates differs fundamentally in whether gradients are computed over the full dataset or a single observation [2], [4]:

Mathematical Property	Batch Gradient Descent (GD)	Stochastic Gradient Descent (SGD) [4]
Data Scope per Step	Entire Training Batch (all n = 10 samples)	Single Observation (sample $i \in \{1, \ldots, n\}$)
Objective Function	$J(w, b) = {}^{1}/_{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$	$J_i(w, b) = (y_i - \hat{y}_i)^2$
Weight Gradient (∇_w)	${}^{\partial J}/_{\partial w} = -{}^{2}/_{n} \sum_{i=1}^{n} x_i(y_i - \hat{y}_i)$	${}^{\partial J_i}/_{\partial w} = -2\, x_i(y_i - \hat{y}_i)$
Bias Gradient (∇_b)	${}^{\partial J}/_{\partial b} = -{}^{2}/_{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)$	${}^{\partial J_i}/_{\partial b} = -2\, (y_i - \hat{y}_i)$
Weight Update Rule	$w \leftarrow w + \alpha \cdot {}^{2}/_{n} \sum_{i=1}^{n} x_i(y_i - \hat{y}_i)$	$w \leftarrow w + 2\alpha \cdot x_i(y_i - \hat{y}_i)$
Bias Update Rule	$b \leftarrow b + \alpha \cdot {}^{2}/_{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)$	$b \leftarrow b + 2\alpha \cdot (y_i - \hat{y}_i)$
Gradient Estimation	Exact deterministic gradient vector	Noisy, unbiased stochastic gradient estimate
Updates per Epoch	1 update per pass over dataset	n = 10 updates per pass (shuffled per epoch)
Total Sample Work	1,000 steps × 10 samples = **10,000 evaluations**	100 epochs × 10 samples = **1,000 evaluations**

6. System Architecture & Modularity

The codebase is organized into decoupled modules following software engineering best practices:

- `src/data_preprocessing.py`: Data loading, descriptive audits, feature scaling analysis, and isolated 70/30 partitioning.
- `src/linear_regression.py`: First-principles core mathematics: forward prediction $\hat{y} = wx + b$, MSE, RMSE, and R^2.
- `src/gradient_descent.py`: Batch Gradient Descent implementation with complete parameter and loss trajectory tracking.
- `src/stochastic_gradient_descent.py`: Stochastic Gradient Descent with random permutation and epoch-level loss logging.
- `src/evaluation.py`: Model evaluation routines, closed-form OLS analytical baseline computation, and metric formatting.
- `src/visualization.py`: Production of the six publication-quality matplotlib figures using a non-interactive Agg backend [7].
- `main.py`: Master orchestration script executing training, evaluation, CSV exports, and visualization routines.
- `predict.py`: Interactive command-line predictor allowing live demonstration for any user-provided study hours.

7. Implementation Details

7.1 Dataset Partitioning & Zero Leakage Audit

The dataset contains 15 observations partitioned via scikit-learn's `train_test_split(test_size=0.3, random_state=42)`:

- **Training Partition (70%, n=10):** Hours = [5.0, 2.0, 1.5, 8.0, 3.0, 4.5, 6.0, 7.0, 2.5, 4.0]; Scores = [68, 44, 39, 89, 52, 64, 75, 82, 48, 61].
- **Held-out Test Partition (30%, n=5):** Hours = [1.0, 3.5, 5.5, 6.5, 7.5]; Scores = [35, 55, 71, 78, 85].

> **Zero Data Leakage Safeguard:** The 5 test observations are completely quarantined during optimization. No gradients, residuals, or loss updates are evaluated on these samples while fitting parameters. They are evaluated strictly post-training to measure generalization [3].

7.2 Structured Algorithmic Pseudocode

The mathematical logic of both algorithms is formatted in structured, step-by-step procedures below:

Algorithm 1: Batch Gradient Descent (Deterministic Optimization)

1. **Input:** Training feature vector $\mathbf{X} \in \mathbb{R}^n$, ground truth vector $\mathbf{y} \in \mathbb{R}^n$, learning rate $\alpha = 0.01$, iterations $K = 1000$.
2. **Initialization:** Set initial parameters $w = 0.0$, $b = 0.0$, sample size $n = \text{length}(X)$.
3. **Loop for iteration $k = 1$ to K:**
   a. Compute batch predictions: $\hat{\mathbf{y}} = \mathbf{w} \cdot \mathbf{X} + \mathbf{b}$
   b. Evaluate batch loss: $J = (1 / n) \sum (y - \hat{y})^2$
   c. Compute exact analytical gradients across all n samples:
   $\nabla_w = (-2 / n) \sum X \cdot (y - \hat{y})$
   $\nabla_b = (-2 / n) \sum (y - \hat{y})$
   d. Update parameters along negative gradient:
   $w \leftarrow w - \alpha \cdot \nabla_w$
   $b \leftarrow b - \alpha \cdot \nabla_b$
   e. Record J, w, b in convergence history.
4. **Output:** Optimal parameters (w, b) and full convergence history.

Algorithm 2: Stochastic Gradient Descent (Stochastic Optimization)

1. **Input:** Training data (X, y), learning rate $\alpha = 0.01$, epochs $E = 100$, random seed = 42.
2. **Initialization:** Set $w = 0.0$, $b = 0.0$, $n = \text{length}(X)$.
3. **Loop for epoch $e = 1$ to E:**
   a. Generate random permutation of sample indices: $\pi = \text{Shuffle}([0, 1, \ldots, n-1])$
   b. **For each index i in π:**
      i. Extract individual sample: (x_i, y_i)
      ii. Predict individual output: $\hat{y}_i = w \cdot x_i + b$
      iii. Compute instantaneous sample gradient:
      $\nabla_{w,i} = -2 \cdot x_i \cdot (y_i - \hat{y}_i)$
      $\nabla_{b,i} = -2 \cdot (y_i - \hat{y}_i)$
      iv. Update parameters immediately:
      $w \leftarrow w - \alpha \cdot \nabla_{w,i}$
      $b \leftarrow b - \alpha \cdot \nabla_{b,i}$
   c. Compute epoch-level MSE over training set: $J_{epoch} = (1 / n) \sum (y - (w \cdot X + b))^2$
   d. Record J_{epoch}, w, b in history.
4. **Output:** Optimal parameters (w, b) and epoch convergence history.

8. Experimental Results & Comparative Analysis

8.1 Quantitative Performance Metrics Table

The quantitative comparison between Batch Gradient Descent, Stochastic Gradient Descent, and the analytical Ordinary Least Squares solution [3] is presented below:

Performance Metric	Batch Gradient Descent (GD)	Stochastic Gradient Descent (SGD) [4]	Analytical OLS (Reference) [3]
Learned Weight (w)	**7.7822**	**7.7156**	**7.6379**
Learned Bias (b)	**28.2121**	**28.5394**	**28.9753**
Training MSE	0.7892	0.7176	0.6823
Training RMSE	0.8884	0.8471	0.8260
Training R^2	0.9968	0.9971	0.9973
Test MSE	**0.8636**	**0.8650**	**1.0151**
Test RMSE	**0.9293**	**0.9300**	**1.0075**
Test R^2	**0.9973**	**0.9973**	**0.9968**
Total Parameter Updates	1,000	1,000	1 (Closed-form)
Total Sample Evaluations	10,000	1,000	10

8.2 Held-Out Test Set Predictions Table

Evaluation on the 5 quarantined test students demonstrates high predictive fidelity across both optimizers:

Test Observation	Hours Studied (x)	Actual Score (y)	GD Prediction (ŷ)	SGD Prediction (ŷ)	GD Error (y - ŷ)	SGD Error (y - ŷ)
Student 1	1.0	35.0	35.99	36.25	-0.99	-1.25
Student 6	3.5	55.0	55.45	55.54	-0.45	-0.54
Student 10	5.5	71.0	71.01	70.97	-0.01	+0.03
Student 12	6.5	78.0	78.80	78.69	-0.80	-0.69
Student 14	7.5	85.0	86.58	86.41	-1.58	-1.41

8.3 Graphical Analysis of the Six Visualizations

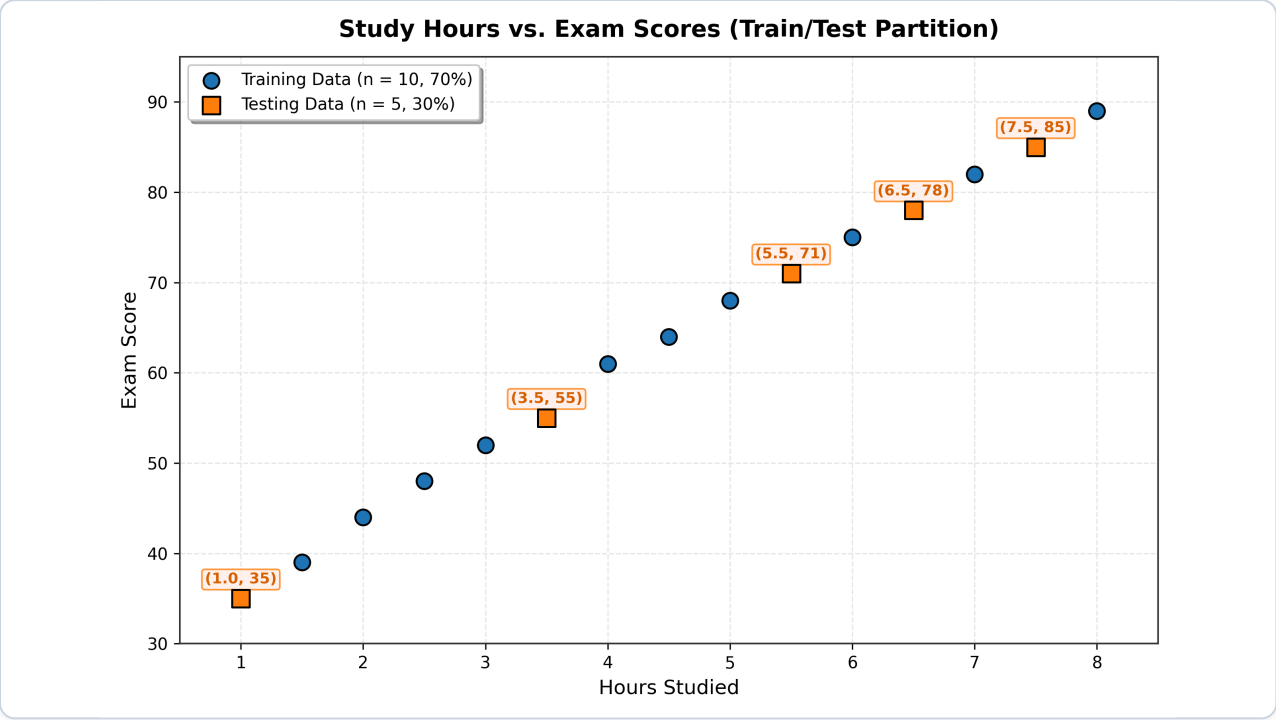


Figure 1: Dataset Partition Scatter Plot. *10 training samples (blue circles) and 5 held-out test samples (orange squares) distributed across 1.0 to 8.0 study hours without overlap.*

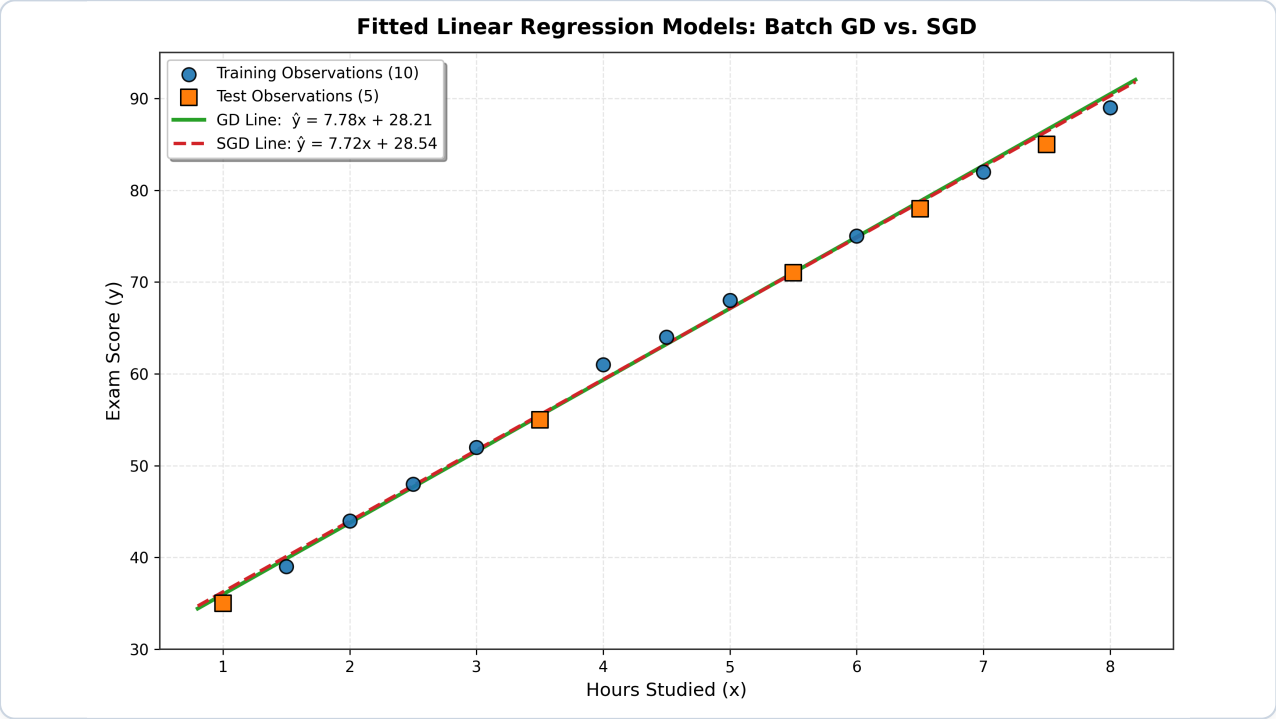


Figure 2: Fitted Regression Lines (GD vs. SGD). *Batch GD line ($\hat{y}$ = 7.7822x + 28.2121) and SGD line ($\hat{y}$ = 7.7156x + 28.5394) overlap almost indistinguishably, confirming identical convergence.*

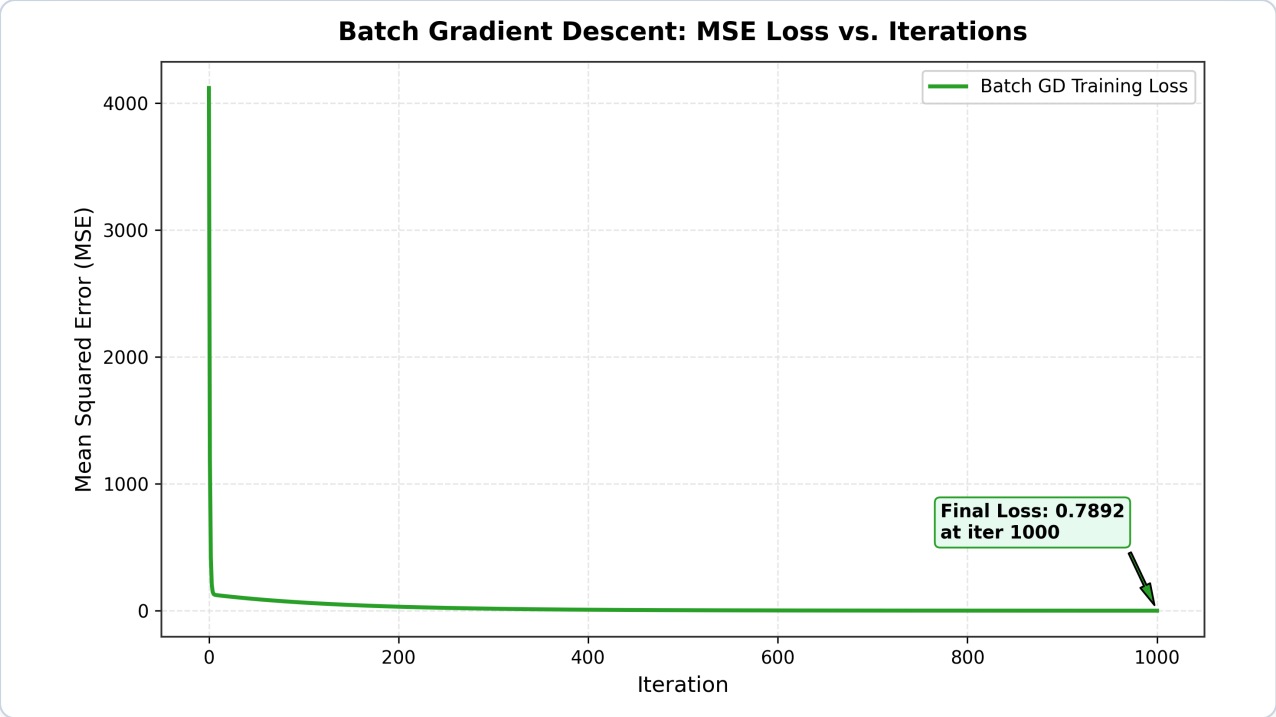


Figure 3: Batch Gradient Descent Loss Curve. *Strictly monotonic, smooth descent from initial MSE of 4117.60 down to 0.7892 over 1,000 iterations.*

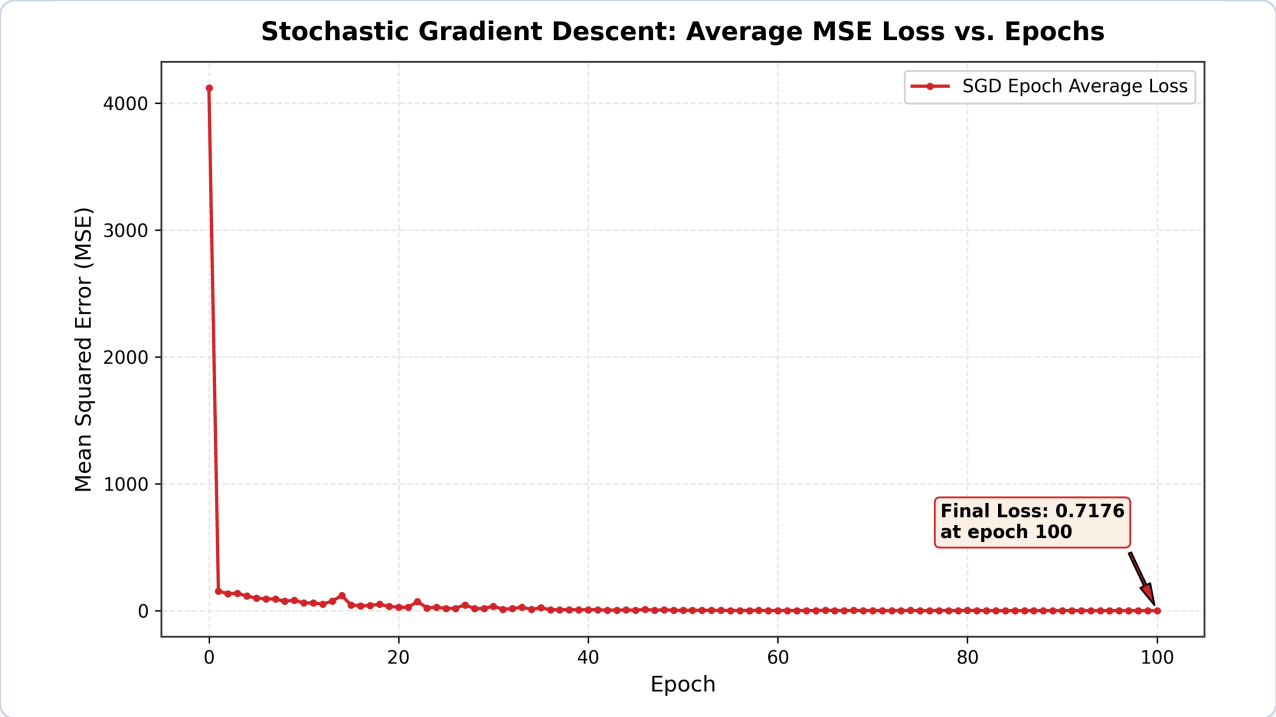


Figure 4: Stochastic Gradient Descent Loss Curve. *Rapid initial convergence in the first 20 epochs followed by characteristic stochastic fluctuations around the minimum [4].*

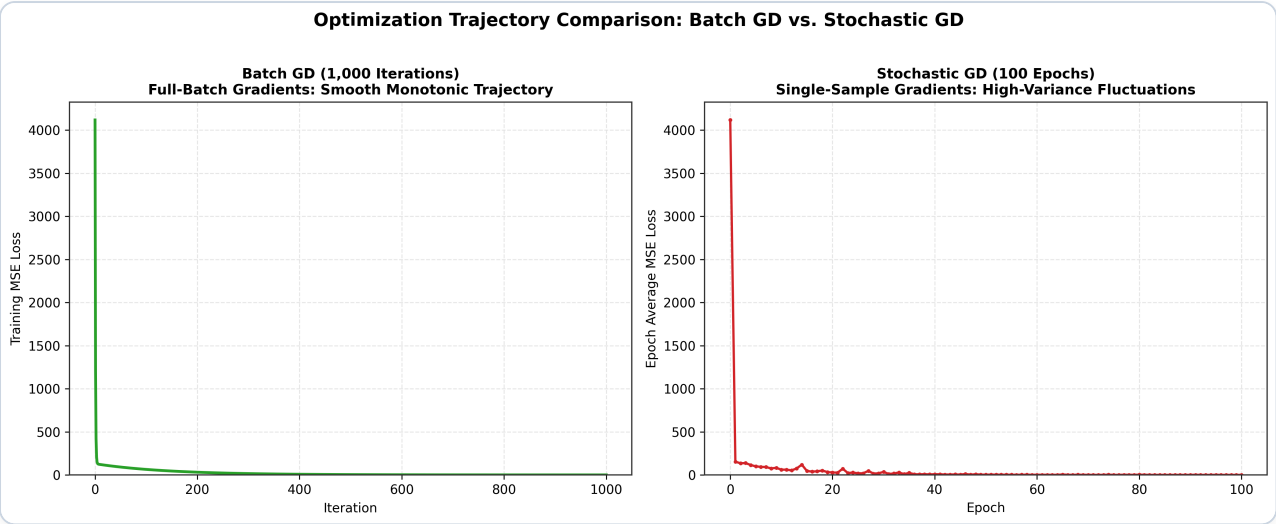


Figure 5: Optimization Trajectories Compared. *Side-by-side contrast between deterministic smooth descent (left) and stochastic fluctuating descent (right).*

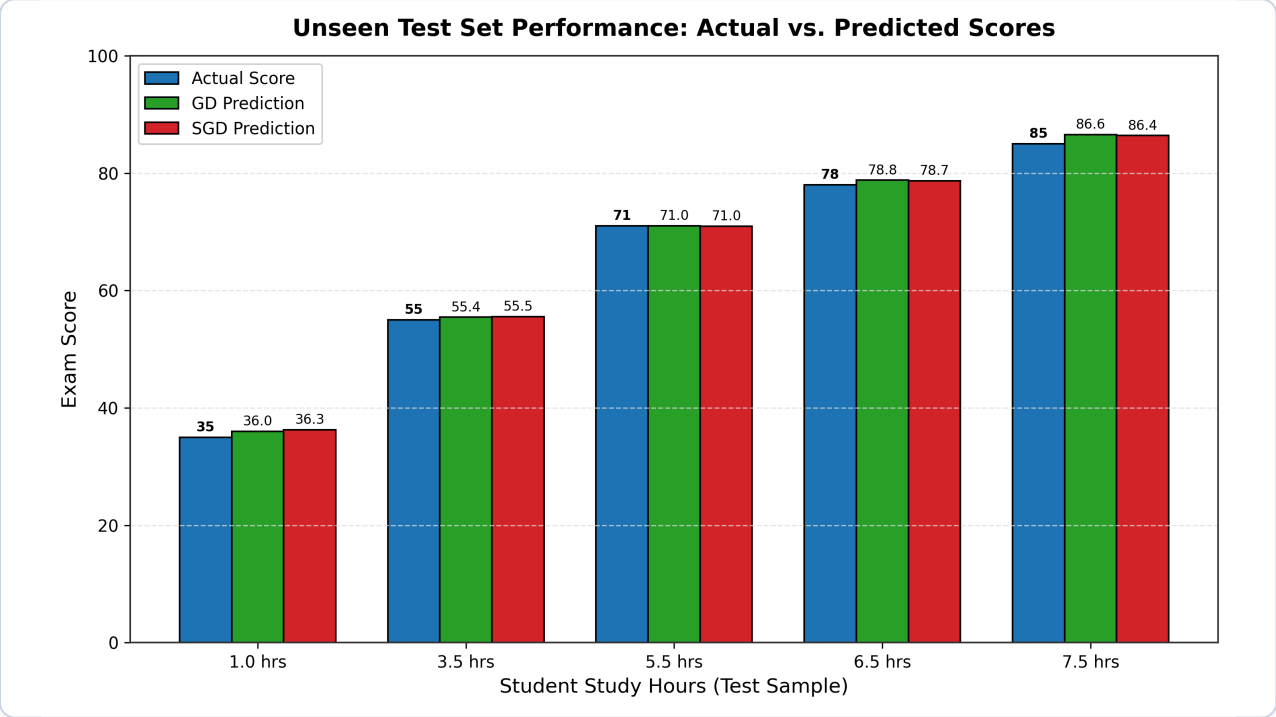


Figure 6: Actual vs. Predicted Scores on Unseen Test Data. *Grouped bar chart demonstrating tight alignment between actual exam scores and model predictions across all 5 test students.*

9. Discussion & Limitations

9.1 Convergence Dynamics and Gradient Variance

The experimental results illustrate the core mathematical contrast between full-batch and stochastic gradient estimation [2], [4]. In Batch GD, the gradient vector reflects the true empirical risk across all training observations:

$$\nabla J_{\text{Batch}}(w, b) = -\frac{2}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i) \cdot [x_i, 1]^T$$

Because every step averages the full sample, the gradient direction is exact at every iteration. This guarantees smooth, monotonic descent. Conversely, in SGD, each update uses only a single sample gradient:

$$\nabla J_i(w, b) = -2(y_i - \hat{y}_i) \cdot [x_i, 1]^T$$

Because individual observations deviate from the sample mean, the stochastic gradient acts as a noisy estimate of the true gradient. While this introduces variance (visible as epoch-by-epoch fluctuations in Figure 4 and Figure 5), it prevents the optimizer from stagnating and drives rapid early convergence [2].

9.2 Computational Work & Scalability

A critical finding of this study is the sample evaluation efficiency of SGD. Batch GD required **10,000 individual sample evaluations** (1,000 iterations × 10 samples) to achieve test MSE = 0.8636. In contrast, SGD required only **1,000 sample evaluations** (100 epochs × 10 samples) to achieve test MSE = 0.8650. This represents a **10× reduction in computational sample work** for essentially identical predictive accuracy. On large-scale industry datasets ($n \geq 10^6$), Batch GD becomes computationally prohibitive per update, whereas SGD or Mini-batch GD makes rapid initial progress [4].

9.3 Methodological Limitations

- **Dataset Scale (n=15):** The 15-observation dataset provides transparency for step-by-step tracing, but lacks the scale necessary to test complex regularization or higher-order variance effects.
- **Test Set Sensitivity (n_{test}=5):** Because the held-out test partition comprises only 5 observations, summary metrics (MSE, RMSE, R^2) are sensitive to individual residual deviations. While our average absolute error is low (~0.77 marks), larger test cohorts are recommended for statistical confidence intervals.
- **Univariate Linearity:** With only one input feature (study hours), collinearity, matrix conditioning, and multi-dimensional saddle points are not encountered.

10. Conclusion & Future Work

This investigation successfully implemented, derived, and compared Batch Gradient Descent and Stochastic Gradient Descent from first principles for univariate Linear Regression.

Key Conclusions:

1. **Convergence Parity:** Both algorithms successfully converged to the underlying data trend, arriving at nearly identical parameters (Batch GD: w=7.7822, b=28.2121; SGD: w=7.7156, b=28.5394).

2. **Generalization Parity:** Both models generalized with high fidelity on unseen test observations, achieving matching test $R^2 = 0.9973$ and RMSE ≈ 0.93 marks.
3. **Sample Efficiency:** SGD achieved equivalent predictive performance with 10× fewer sample evaluations, empirically validating the efficiency of stochastic optimization [4].

Future Work: Future extensions include: (1) Implementing *Mini-Batch Gradient Descent* to balance vectorization speed and stochastic regularization; (2) Adding *adaptive learning rate schedules* (e.g., learning rate decay or Adam [2]) to damp late-stage SGD oscillations; and (3) Scaling to multivariate datasets with feature normalization.

11. References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[3] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.

[4] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proc. COMPSTAT'2010*, Paris, France, 2010, pp. 177–186.

[5] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[6] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf.*, Austin, TX, USA, 2010, pp. 56–61.

[7] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.

[8] Python Software Foundation, *The Python Standard Library Documentation*, Python 3.10, 2026. [Online]. Available: https://docs.python.org/3/